2.2 Methods of error detection

2.2.1 The need to check for errors

When data is transmitted, there is always a risk that it may be corrupted, lost or even gained.

Errors can occur during data transmission due to:

- interference (all types of cable can suffer from electrical interference, which can cause data to be corrupted or even lost)
- problems during packet switching (this can lead to data loss – or it is even possible to gain data!)
- skewing of data (this occurs during parallel data transmission and can cause data corruption if the bits arrive out of synchronisation).

Checking for errors is important since computers are unable to understand text, for example, if the words are not recognised by its built-in dictionary. Look at the following example of some corrupted text:

Can you raed tihs?

"I cnduo't bvleiee taht I culod aulaclty uesdtannrd waht I was rdnaieg. Unisg the icndeblire pweor of the hmuan mnid, aocdcrnig to rseecrah at Cmabridge Uinervtisy, it dseno't mttaer in waht oderr the lterets in a wrod are, the olny irpoamtnt tihng is taht the frsit and lsat ltteer be in the rhgit pclae. The rset can be a taotl mses and you can sitll raed it whoutit a pboerlm.

Tihs is bucseae the huamn mnid deos not raed ervey ltteer by istlef, but the wrod as a wlohe.

Aaznmig, huh? Yeah and I awlyas tghhuot slelinpg was ipmorantt! See if yuor fdreins can raed tihs too"

(from an unknown source at Cambridge University)

▲ **Figure 2.13** Example of data corruption on a message

Whilst you probably had little problem understanding this text, a computer would be unable to make any sense of it. Data corruption is therefore a very real problem to a computer. Figure 2.13 could be the result of some data corruption following transmission which would make the text unintelligible to a computer. This is why error checking is such an important part of computer technology. The following section considers a number of ways that can be used to check for errors, so that you don't end up with text as shown in Figure 2.13 above!

There are a number of ways data can be checked for errors following transmission:

- parity checks
- checksum
- echo check.

2.2.2 Parity checks, checksum and echo checks

Parity checks

Parity checking is one method used to check whether data has been changed or corrupted following data transmission. This method is based on the number of 1-bits in a byte of data.

The parity can be either called **EVEN** (that is, an even number of 1-bits in the byte) or **ODD** (that is, an odd number of 1-bits in the byte). One of the bits in the byte (usually the most significant bit or left-most bit) is reserved for a **parity bit**. The parity bit is set according to whether the parity being used is even or odd. For example, consider the byte:

	1	1	0	1	1	0	0

parity bit

Advice

Note for the 7 bit number 1110000, the even parity bit would be 1 and the odd parity bit would be 0. The parity bit can be set as a 1 or a 0 for either choice of parity – it just depends on how many 1s are in the byte.

In this example, if the byte is using even parity, then the parity bit needs to be set to 0, since there is already an even number of 1-bits in the byte (four 1-bits). We thus get:

0	1	1	0	1	1	0	0

parity bit

In this example, if the byte is using odd parity, then the parity bit needs to be set to 1, since we need to have an odd number of 1-bits in the byte. We thus get:

1	1	1	0	1	1	0	0

parity bit

Before data is transferred, an agreement is made between sender and receiver regarding which type of parity is being used. Parity checks are therefore being used as a type of transmission protocol.

Activity 2.4

Find the parity bits for each of the following bytes:

	1	1	0	1	1	0	1	even parity being used
	0	0	0	1	1	1	1	even parity being used
	0	1	1	1	0	0	0	even parity being used
	1	1	1	0	1	0	0	odd parity being used
	1	0	1	1	0	1	1	odd parity being used
	1	1	1	1	1	1	0	even parity being used
	1	1	1	1	1	1	0	odd parity being used
	1	1	0	1	0	0	0	odd parity being used
	0	0	0	0	1	1	1	even parity being used
	1	1	1	1	1	1	1	odd parity being used

If a byte has been transmitted from 'A' to 'B', and if even parity is used, an error would be flagged if the byte now had an odd number of 1-bits at the receiver's end. For example (assuming even parity is being used):

byte being sent:	0	1	0	1	1	1	0	0
	parity bit							
byte being received:	0	1	0	0	1	1	0	0
	parity bit							

In this case, the byte received has three 1-bits, which means it now has odd parity; while the sender's byte was using even parity (four 1-bits). This means an error has occurred during the transmission of the byte. The error is detected by the recipient's computer re-calculating the parity of the byte sent. If even parity had been agreed between sender and receiver, then a change in parity in the received byte indicates that a transmission error has occurred.

Activity 2.5

1 Which of the following received bytes indicate an error has occurred following data transmission?

1	1	1	0	1	1	0	1	even parity being used
0	1	0	0	1	1	1	1	even parity being used
0	0	1	1	1	0	0	0	even parity being used
1	1	1	1	0	1	0	0	odd parity being used
1	1	0	1	1	0	1	1	odd parity being used
1	1	1	1	1	1	1	1	odd parity being used
0	0	0	0	0	0	0	0	even parity being used
1	1	1	0	0	0	0	0	odd parity being used
0	1	0	1	0	1	0	1	even parity being used
1	1	1	0	0	0	1	1	odd parity being used

2 In each case, in **question 1**, where an error occurred, can you work out which bit in the byte was changed during data transmission?

If two of the bits change value following data transmission, it may be impossible to locate the error using parity checking.

Let us imagine we are transmitting the following byte, using even parity:

0	**1**	**0**	**1**	**1**	**1**	**0**	**0**

Suppose more than one bit has been modified during data transmission. This means the byte could have reached the destination as any of the following:

0	**1**	**1**	**1**	**1**	**1**	**0**	**1**	six 1-bits

0	1	0	1	0	0	0	0	two 1-bits

0	1	0	1	0	1	1	0	four 1-bits

In all these cases, the byte has clearly been corrupted, but the bytes have retained even parity. Therefore, no error would be flagged in spite of the obvious errors in transmission. Clearly it will be necessary to have other ways to complement parity when it comes to error checking to ensure errors are never missed. One such method is called checksum – see the next section.

You should have concluded that *any* of the bits in question 2 (Activity 2.5) could have been changed where there was a transmission error. Therefore, even though an error has been flagged, it is impossible to know **exactly** which bit is in error.

One of the ways round this problem is to use **parity blocks**. In this method, a block of data is sent and the number of 1-bits are totalled horizontally and vertically (in other words, a parity check is done in both horizontal and vertical directions). As the following example shows, this method not only identifies that an error has occurred but also indicates where the error is.

Example 1

In this example, nine bytes of data have been transmitted. Agreement has been made that **even parity** will be used. Another byte, known as the **parity byte**, has also been sent. This byte consists entirely of the parity bits produced by the vertical parity check. The parity byte also indicates the end of the block of data.

Table 2.3 shows how the data arrived at the receiving end. It is now necessary to check the parity of each byte horizontally (bytes 1 to 9) and vertically (columns 1 to 8). Each row and column where the parity has changed from even to odd should be flagged:

▼ **Table 2.3** Parity block showing nine bytes and parity byte

	Parity bit	Bit 2	Bit 3	Bit 4	Bit 5	Bit 6	Bit 7	Bit 8
Byte 1	1	1	1	1	0	1	1	0
Byte 2	1	0	0	1	0	1	0	1
Byte 3	0	1	1	1	1	1	1	0
Byte 4	1	0	0	0	0	0	1	0
Byte 5	0	1	1	0	1	0	0	1
Byte 6	1	0	0	0	1	0	0	0
Byte 7	1	0	1	0	1	1	1	1
Byte 8	0	0	0	1	1	0	1	0
Byte 9	0	0	0	1	0	0	1	0
Parity byte	1	1	0	1	0	0	0	1

A careful study of Table 2.3 shows the following:

- byte 8 (row 8) now has incorrect parity (there are three 1-bits)
- bit 5 (column 5) also now has incorrect parity (there are five 1-bits).

First of all, the table shows that an error has occurred following data transmission (there has been a change in parity in one of the bytes).

Secondly, at the intersection of row 8 and column 5, the position of the incorrect bit value (which caused the error) can be found. The 1-bit at this intersection should be a 0-bit; this means that byte 8 should have been:

0	0	0	1	0	0	1	0

which would also correct column 5 giving an even vertical parity (now has four 1-bits).

This byte could therefore be corrected automatically as shown above, or an error message could be relayed back to the sender asking them to re-transmit the block of data.

Activity 2.6

1 The following block of data was received after transmission from a remote computer; **odd parity** was being used by both sender and receiver. One of the bits has been changed during the transmission stage. Locate where this error is and suggest a corrected byte value:

	Parity bit	Bit 2	Bit 3	Bit 4	Bit 5	Bit 6	Bit 7	Bit 8
Byte 1	0	1	1	0	0	0	1	0
Byte 2	1	0	1	1	1	1	1	1
Byte 3	1	0	0	1	1	0	0	0
Byte 4	0	1	1	0	1	0	1	0
Byte 5	1	1	1	0	0	1	1	0
Byte 6	1	0	0	0	0	1	0	1
Byte 7	0	1	1	1	0	0	0	0
Byte 8	0	0	0	0	0	0	0	1
Byte 9	0	1	1	1	1	0	1	0
Parity byte	1	0	1	1	1	1	0	0

2 The following block of data was received after transmission from a remote computer. Even parity was being used by both sender and receiver. One of the bytes has been changed during the transmission stage. Locate where this error is and suggest a corrected byte value.

	Parity bit	Bit 2	Bit 3	Bit 4	Bit 5	Bit 6	Bit 7	Bit 8
Byte 1	1	1	0	0	0	0	0	0
Byte 2	0	0	1	1	1	1	0	0
Byte 3	0	1	0	0	0	1	1	1
Byte 4	1	0	1	0	1	1	1	1
Byte 5	0	0	0	1	0	0	0	1
Byte 6	0	0	1	1	1	1	1	1
Byte 7	1	0	1	1	0	1	0	0
Byte 8	0	1	0	1	0	0	0	1
Byte 9	1	1	1	0	0	1	0	0
Parity byte	0	0	1	1	1	0	0	1

Checksum

A **checksum** is a method used to check if data has been changed or corrupted following data transmission. Data is sent in blocks, and an additional value, called the checksum, is sent at the end of the block of data.

The checksum process is as follows:

- when a block of data is about to be transmitted, the checksum is calculated from the block of data
- the calculation is done using an agreed algorithm (this algorithm has been agreed by sender and receiver)
- the checksum is then transmitted with the block of data
- at the receiving end, the checksum is recalculated by the computer using the block of data (the agreed algorithm is used to find the checksum)
- the re-calculated checksum is then compared to the checksum sent with the data block
- if the two checksums are the same, then no transmission errors have occurred; otherwise a request is made to re-send the block of data.

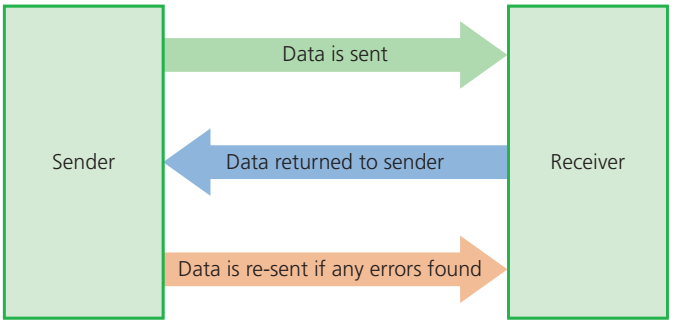
Echo check

With **echo check**, when data is sent to another device, this data is sent back again to the sender. The sender's computer compares the two sets of data to check if any errors occurred during the transmission process.

As you will have no doubt worked out, this isn't very reliable. If the two sets of data are different, it isn't known whether the error occurred when sending the data in the first place, or if the error occurred when sending the data back for checking.

However, if no errors occurred, then it is another way to check that the data was transmitted correctly. In summary:

- a copy of the data is sent back to the sender
- the returned data is compared with the original data by the sender's computer
- if there are no differences, then the data was sent without error
- if the two sets of data are different, then an error occurred at some stage during the data transmission.



▲ **Figure 2.14** Echo check diagram

2.2.3 Check digits

A **check digit** is the final digit included in a code; it is calculated from all the other digits in the code. Check digits are used for barcodes on products, such as International Standard Book Numbers (ISBN) and Vehicle Identification Numbers

(VIN). Check digits are used to identify errors in *data entry* caused by mis-typing or mis-scanning a barcode. They can usually detect the following types of error:

- an incorrect digit entered, for example 5327 entered instead of 5307
- transposition errors where two numbers have changed order, for example 5037 instead of 5307
- omitted or extra digits, for example 537 instead of 5307 or 53107 instead of 5307
- phonetic errors, for example 13 (thirteen), instead of 30 (thirty).



▲ **Figure 2.15** Sample barcode (ISBN 13 code with check digit)

There are a number of different methods used to generate a check digit. Two common methods will be considered here:

- ISBN 13
- Modulo-11

Example 1: ISBN 13

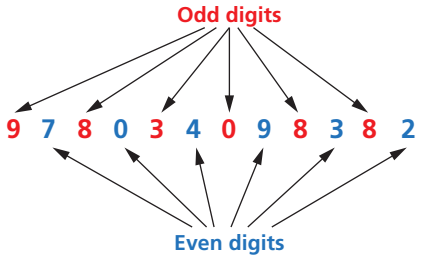
The check digit in ISBN 13 is the thirteenth digit in the number. We will now consider two different calculations. The first calculation is the generation of the check digit. The second calculation is a verification of the check digit (that is, a recalculation).

Calculation 1 – Generation of the check digit from the other 12 digits in a number

The following algorithm generates the check digit from the 12 other digits:

1. add all the odd numbered digits together
2. add all the even numbered digits together and multiply the result by 3
3. add the results from 1 and 2 together and divide by 10
4. take the remainder, if it is zero then use this value, otherwise subtract the remainder from 10 to find the check digit.

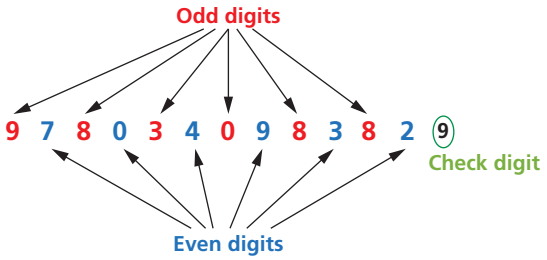
Using the ISBN **9 7 8 0 3 4 0 9 8 3 8 2** (note this is the same ISBN as in Figure 2.15):



▲ **Figure 2.16** ISBN (no check digit)

1. **9 + 8 + 3 + 0 + 8 + 8 = 36**
2. **3 × (7 + 0 + 4 + 9 + 3 + 2) = 75**
3. (**36** + **75**)/10 = 111/10 = 11 remainder 1
4. 10 – 1 = **9** the check digit

So we end up with the following thirteen-digit number (which matches the number shown in Figure 2.15):



▲ **Figure 2.17** ISBN (including the check digit)

Calculation 2 – Re-calculation of the check digit from the thirteen-digit number (which now includes the check digit)

To check that an ISBN 13-digit code is correct, including its check digit, a similar process is followed:

1 add all the odd numbered digits together, **including** the check digit
2 add all the even number of digits together and multiply the result by 3
3 add the results from 1 and 2 together and divide by 10
4 the number is correct if the remainder is zero.

Using the ISBN **9 7 8 0 3 4 0 9 8 3 8 2 9** (including its check digit) from Figure 2.17:

1 **9 + 8 + 3 + 0 + 8 + 8 + 9 = 45**
2 **3 × (7 + 0 + 4 + 9 + 3 + 2) = 75**
3 (45 + 75)/10 = 120/10 = 12 remainder **0**
4 remainder is 0, therefore number is correct.

Advice

You will not need to remember the steps shown in these algorithms; the steps will be given to you, but it is important that you understand how to use an algorithm to calculate or verify check digits.

? Example 2: Modulo-11

The modulo-11 method can have varying lengths of number which makes it suitable for many applications, such as product codes or VINs. The first calculation is the generation of the check digit. The second calculation is a verification of the check digit (that is, a recalculation).

Calculation 1 – Generation of the check digit from the other digits in a number

(In this example, we will assume the original number contained only 7 digits.)

The following algorithm generates the check digit from the other 7 digits:

1 each digit in the number is given a weighting of 8, 7, 6, 5, 4, 3 or 2 starting from the left (weightings start from 8 since the number will become eight-digit when the check digit is added)
2 the digit is multiplied by its weighting and then each value is added to make a total
3 the total is divided by 11
4 the remainder is then subtracted from 11 to find the check digit (note if the remainder is 10 then the check digit 'X' is used).

The example to be used has the following seven-digit number:

1 7-digit number: 4 1 5 6 7 1 0

weighting values: 8 7 6 5 4 3 2

2 sum: (8 × 4) + (7 × 1) + (6 × 5) + (5 × 6) + (4 × 7) + (3 × 1) + (2 × 0)
= 32 + 7 + 30 + 30 + 28 + 3 + 0
total = 130
3 divide total by 11: 130/11 = 11 remainder 9
4 subtract remainder from 11: 11 – 9 = 2 (check digit)

So we end up with the following eight-digit: 4 1 5 6 7 1 0 2

Calculation 2 – Re-calculation of the check digit from the eight-digit number (which now includes the check digit)

To check that the eight-digit number is correct, including its check digit, a similar process is followed:

1 each digit in the number is given a weighting of 8, 7, 6, 5, 4, 3, 2 or 1 starting from the left
2 the digit is multiplied by its weighting and then each value is added to make a total
3 the total is divided by 11
4 the number is correct if the remainder is zero

Using the 8-digit number: 4 1 5 6 7 1 0 2
1 weighting values: 8 7 6 5 4 3 2 1
2 sum: (8 × 4) + (7 × 1) + (6 × 5) + (5 × 6) + (4 × 7) + (3 × 1) + (2 × 0) + (1 × 2)
= 32 + 7 + 30 + 30 + 28 + 3 + 0 + 2
total = 132
3 divide total by 11: 132/11 = 12 remainder 0
4 remainder is 0, therefore number is correct

Activity 2.7

1 Using the algorithm for ISBN-13 calculate the check digit for:

`978151045759`

2 Find the check digits for the following numbers using **both** modulo-11 and ISBN 13 methods:
i 2 1 3 1 1 1 0 0 0 4 2 8
ii 9 0 9 8 1 2 1 2 3 5 4 4

2.2.4 Automatic Repeat Requests (ARQs)

We have already considered parity checks and echo checks as methods to verify that data has arrived at its destination unchanged. An **Automatic Repeat Request (ARQ)** is a third way used to check data following data transmission. This method can best be summarised as follows:

- ARQ uses positive and negative **acknowledgements** (messages sent to the receiver indicating that data has/has not been received correctly) and **timeout** (this is the time interval allowed to elapse before an acknowledgement is received)
- the receiving device receives an error detection code as part of the data transmission (this is typically a Cyclic Redundancy Check – refer to Section 2.1.1); this is used to detect whether the received data contains any transmission errors
- if no error is detected, a positive acknowledgement is sent back to the sending device
- however, if an error is detected, the receiving device now sends a negative acknowledgement to the sending device and requests re-transmission of the data
- a time-out is used by the sending device by waiting a pre-determined amount of time ….
- … and if no acknowledgement of any type has been received by the sending device within this time limit, it automatically re-sends the data until a positive acknowledgement is received ….
- … or until a pre-determined number of re-transmissions has taken place
- ARQ is often used by mobile phone networks to guarantee data integrity.